
Implementing Linear Regression Using a Multi-Layer Neural Network — From Scratch

A NumPy-only feed-forward regressor trained on the California Housing dataset, benchmarked across three hand-coded optimizers, two learning rates, and two bonus configurations.

1. Introduction & Objective

This report documents the implementation of a **linear regression task using a multi-layer neural network built entirely from scratch** — using only NumPy for all mathematical operations, with no pre-built model such as scikit-learn's LinearRegression/MLPRegressor, PyTorch, or TensorFlow. The network is trained on the **California Housing** dataset to predict MedHouseVal (median house value) from two selected input features: MedInc (median income) and HouseAge (median house age).

The required architecture — Input (2) → Hidden 1 (5, ReLU) → Hidden 2 (3, ReLU) → Output (1, Linear) — was implemented with manual forward propagation, a manually derived backpropagation routine, and three optimizers coded from first principles: basic Gradient Descent, Momentum, and Adam. The main experiments performed are:

- A learning-rate comparison (0.01 vs 0.001) using basic Gradient Descent.
- An optimizer comparison — Gradient Descent, Momentum, and Adam — at a fixed learning rate.
- A combined 6-configuration grid over all optimizer × learning-rate pairs, used to identify the best overall setting.
- Bonus 1: adding a third hidden layer (2–5–3–2–1) to test the effect of extra depth.
- Bonus 2: L2 weight regularization ($\lambda = 0.001$) to test its effect on generalization.

2. Dataset & Preprocessing

The California Housing dataset was loaded via `sklearn.datasets.fetch_california_housing`, giving **20,640 rows and 9 columns**. Only two columns were used as model inputs — MedInc and HouseAge — with MedHouseVal as the regression target, giving feature matrix X of shape (20640, 2) and target y of shape (20640, 1).

Statistic	MedInc (input)	HouseAge (input)	MedHouseVal (target)
Mean	3.8707	28.6395	2.0686
Std. Dev.	1.8998	12.5856	1.1540
Min	0.4999	1.0000	0.1500
Max	15.0001	52.0000	5.0000

Table 2.1 — Descriptive statistics of the raw (pre-normalization) features and target, computed over all 20,640 rows.

The data was split **80:20 into training and test sets** using a seeded random permutation (`numpy.random.default_rng(42)`), giving **16,512 training samples and 4,128 test samples**. Both input features were then standardized (z-score) using the **mean and standard deviation of the training split only** — the same statistics were re-used to transform the test split, preventing test-set information from leaking into training. The target was kept on its original scale since the task is direct regression. After normalization, the training features were confirmed to have mean ≈ 0 and standard deviation = 1.

3. Neural Network Architecture

The network is a fully-connected feed-forward regressor with two hidden layers, matching the assignment specification exactly:

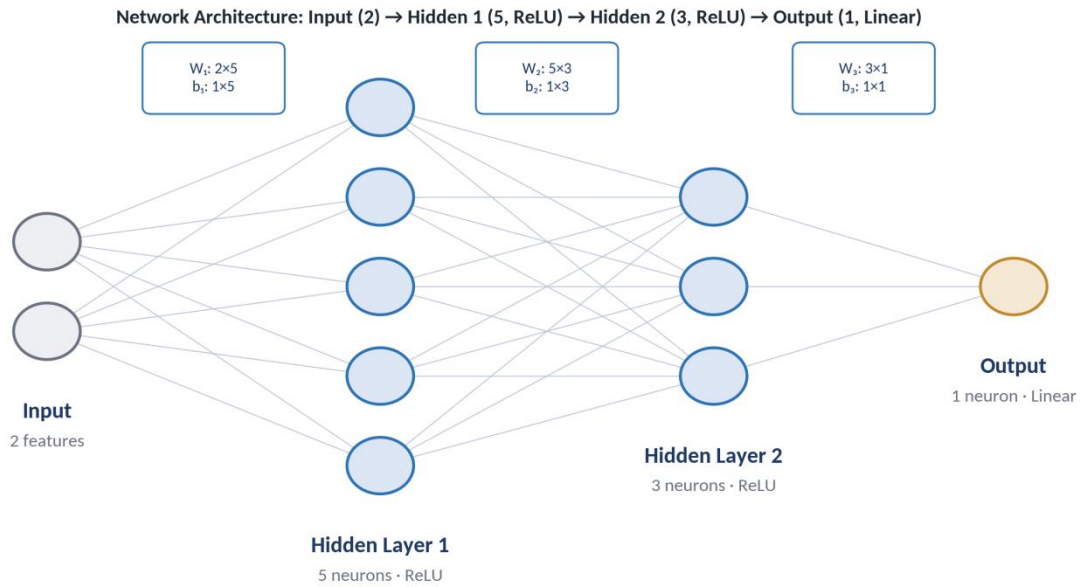


Figure 1 — Network architecture: Input (2) → Hidden 1 (5, ReLU) → Hidden 2 (3, ReLU) → Output (1, Linear).

Layer	Neurons	Activation	Weight Shape	Bias Shape
Input	2	—	—	—
Hidden 1	5	ReLU	2×5	1×5
Hidden 2	3	ReLU	5×3	1×3
Output	1	Linear	3×1	1×1

Table 3.1 — Layer-by-layer neuron counts, activations, and parameter shapes.

Each weight matrix has shape (fan_in, fan_out) because the forward pass computes $Z = A @ W + b$ with A of shape (n, fan_in) — so W must be (fan_in, fan_out) for the matrix product to yield (n, fan_out). This gives the 2×5 , 5×3 and 3×1 shapes above, with one bias row per layer broadcast across all n samples. Weights are He-initialized ($N(0,1) \times \sqrt{2/\text{fan_in}}$) — suited to ReLU hidden layers — and biases start at zero, using a fixed seed (42) so every experiment starts from identical initial parameters.

4. Implementation & Code Flow

Every epoch of training follows the same nine-step loop, implemented without any autograd or ML framework:

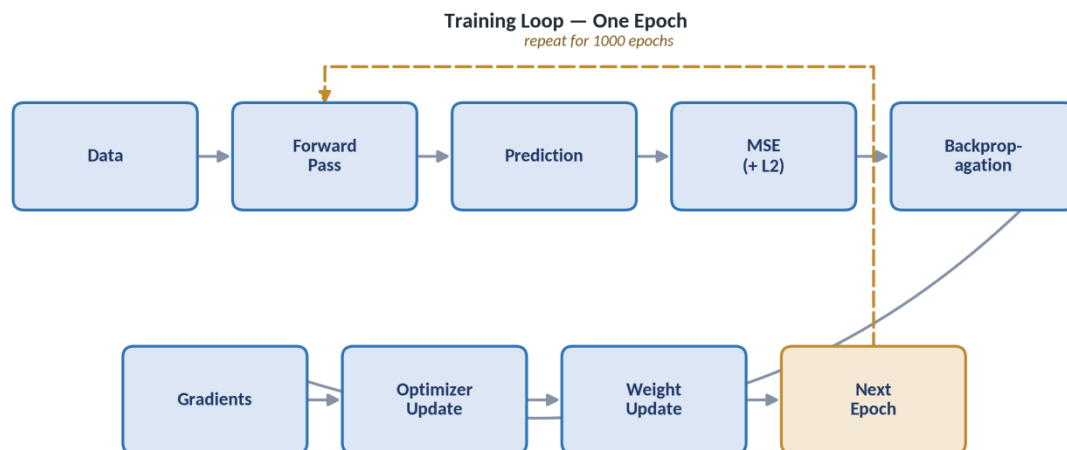


Figure 2 — One training epoch: data flows forward to a prediction and a loss, then gradients flow backward and update the weights before the loop repeats.

4.1 Weight Initialization

The `NeuralNetwork` class builds one weight matrix and bias vector per layer using He initialization:

```
W = rng.standard_normal((fan_in, fan_out)) * np.sqrt(2.0 / fan_in)
b = np.zeros((1, fan_out))
```

4.2 Forward Propagation

The `forward()` method applies a linear transform at every layer, followed by ReLU on the two hidden layers and a linear (identity) activation on the output layer:

```
Z = A @ self.W[layer] + self.b[layer]
A = self.relu(Z) if layer < num_layers - 1 else Z
```

For example, $X @ W_1 + b_1$ transforms an $(n, 2)$ input batch into $(n, 5)$ hidden-layer values; the same pattern then maps $(n, 5) \rightarrow (n, 3) \rightarrow (n, 1)$ through the remaining two layers.

4.3 Loss Function

Mean Squared Error is computed directly, with an optional L2 penalty added on top for the regularization experiment:

```
data_loss = np.mean((y_true - y_pred) ** 2)
reg_loss = (l2_lambda / (2 * n)) * sum(np.sum(W ** 2) for W in weights)
```

4.4 Manual Backpropagation

The `backward()` function walks the layers in reverse, applying the chain rule by hand. The output layer is linear, so its local gradient is just the MSE derivative; hidden layers multiply by the ReLU derivative:

```
dA = (2.0 / n) * (y_pred - y)          # dLoss/dPrediction
dZ = dA * model.relu_derivative(Z)      # hidden layers only
dW[layer] = A_prev.T @ dZ              # weight gradient
db[layer] = np.sum(dZ, axis=0, keepdims=True) # bias gradient
```

4.5 Optimizers

Three update rules are implemented as small classes sharing a common interface, each consuming the gradients produced above:

- Gradient Descent — $W \leftarrow W - lr * dW$ (no state between epochs).
- Momentum ($\beta = 0.9$) — maintains a velocity term $v = \beta \cdot v + dW$, then $W \leftarrow W - lr * v$.
- Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 1e-8$) — tracks bias-corrected first and second moment estimates and scales each parameter's step individually.

4.6 Training Loop & Experiment Runner

The `train_model()` function repeats `forward` $\rightarrow$ `loss` $\rightarrow$ `backward` $\rightarrow$ `optimizer.update()` for the requested number of epochs (1000 throughout this study) on the full training batch, logging both train and test loss every epoch. A helper, `run_experiment()`, builds a fresh network from the same seed (42) for every configuration — this ensures optimizers and learning rates are compared from identical starting weights, so differences in the results below reflect the update rule, not a lucky initialization.

5. Experimental Setup

All experiments share the same architecture, data split, and epoch budget unless stated otherwise; only the optimizer, learning rate, network depth, or L2 penalty is varied between runs.

Parameter	Setting
Epochs	1000 (full-batch gradient descent every epoch)
Learning rates tested	0.01, 0.001
Optimizers tested	Gradient Descent, Momentum ($\beta = 0.9$), Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$)
Loss function	MSE, with optional L2 penalty
Main architecture	2 – 5 – 3 – 1 (He initialization, seed = 42)
Bonus architecture	2 – 5 – 3 – 2 – 1 (extra ReLU hidden layer, 2 neurons)

Parameter	Setting
Bonus regularization	L2, $\lambda = 0.001$ (compared against $\lambda = 0$)
Train / test split	16,512 / 4,128 samples (80 / 20, seed = 42)
Evaluation metric	Test MSE (RMSE also reported for the best model)

Table 5.1 — Fixed experimental configuration used across Sections 6–7.

6. Results & Analysis

6.1 Learning Rate Comparison

Basic Gradient Descent was trained for 1000 epochs on the 2–5–3–1 network at two learning rates:

Learning Rate	Final Train Loss (epoch 1000)	Test MSE (epoch 1000)
0.01	0.640194	0.648101
0.001	0.887155	0.897964

Table 6.1 — Gradient Descent, learning-rate comparison (2–5–3–1 architecture).

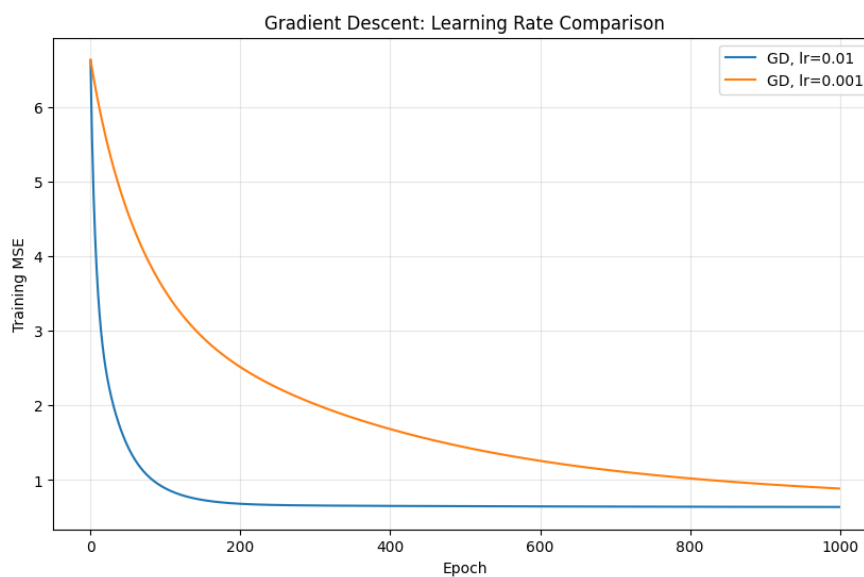


Figure 3 — Training MSE vs. epoch for Gradient Descent at $lr = 0.01$ and $lr = 0.001$.

$lr = 0.01$ converges markedly faster: it drops below a training MSE of 1.0 within roughly 150–200 epochs and flattens out near 0.64 by epoch ≈ 400 , with no visible instability or oscillation in the curve. **$lr = 0.001$** is still descending steadily at epoch 1000 and has clearly not converged within this epoch budget, finishing at a training loss of 0.887 versus 0.640 for the larger rate. For this network and epoch budget, 0.01 is the better choice — it reaches a lower loss faster with no sign of the divergence a too-large rate would cause.

6.2 Optimizer Comparison

Gradient Descent, Momentum, and Adam were each trained for 1000 epochs at a fixed learning rate of 0.001, from identical initial weights:

Optimizer	Learning Rate	Final Train Loss	Test MSE
Momentum	0.001	0.639867	0.647937
Adam	0.001	0.666758	0.683425
Gradient Descent	0.001	0.887155	0.897964

Table 6.2 — Optimizer comparison at $lr = 0.001$, sorted by test MSE.

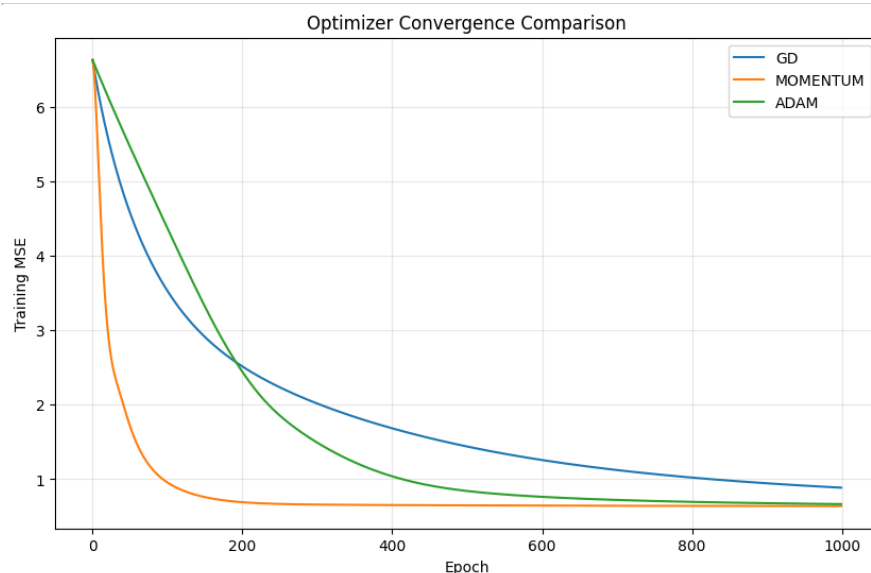


Figure 4 — Training MSE vs. epoch for Gradient Descent, Momentum, and Adam ($lr = 0.001$).

Momentum converges fastest and reaches the lowest loss at this learning rate — its velocity term accelerates early progress, flattening near epoch ≈ 150 . **Adam** descends more slowly over the first ≈ 250 epochs while its bias-corrected moment estimates build up, but still finishes well ahead of plain GD by epoch 1000. **Gradient Descent** is the slowest of the three and has not converged within 1000 epochs at this rate.

Extending this comparison to all six optimizer $\times$ learning-rate pairs (Table 6.3) shows the best individual result was **Momentum at $lr = 0.01$** , reaching a test MSE of **0.619313** — the lowest of any configuration tested. This configuration is used for the predicted-vs-actual evaluation below.

Rank	Configuration	Final Train Loss	Test MSE
1	Momentum, $lr = 0.01$	0.610460	0.619313
2	Adam, $lr = 0.01$	0.614773	0.625793
3	Momentum, $lr = 0.001$	0.639867	0.647937
4	Gradient Descent, $lr = 0.01$	0.640194	0.648101
5	Adam, $lr = 0.001$	0.666758	0.683425
6	Gradient Descent, $lr = 0.001$	0.887155	0.897964

Table 6.3 — All six optimizer $\times$ learning-rate combinations, ranked by test MSE.

6.3 Predicted vs. Actual

Using the best configuration found above (Momentum, $lr = 0.01$), the final evaluation on the held-out test set gives:

Metric	Value
Test MSE	0.619313
Test RMSE	0.786964

Table 6.4 — Final test-set error for the best configuration (Momentum, $lr = 0.01$).

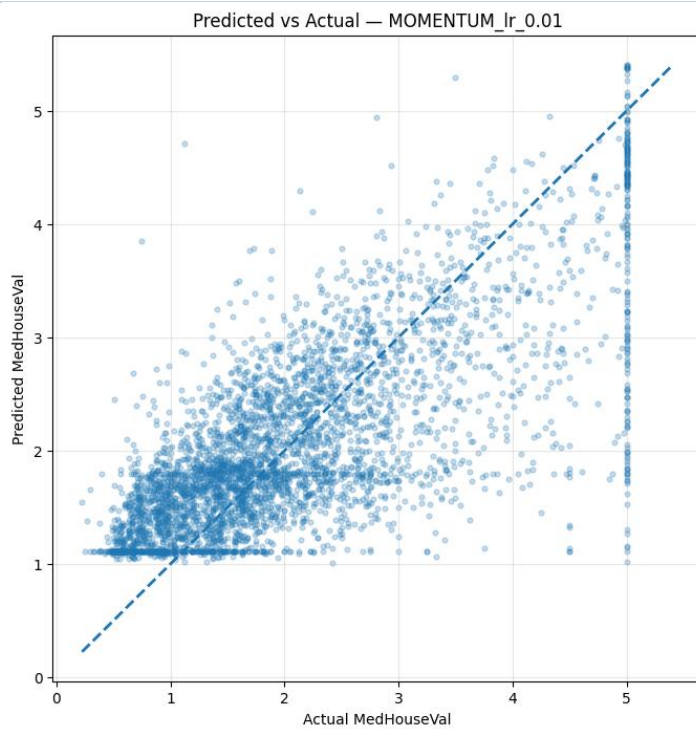


Figure 5 — Predicted vs. actual MedHouseVal on the test set; the dashed line marks perfect prediction ($y = x$).

Points broadly track the diagonal, showing the model captures the general positive relationship between income/age and house value, but the spread is substantial — an RMSE of ≈ 0.79 against a target ranging from 0.15 to 5.0 (mean 2.07) is a sizeable error band relative to the scale of the variable. Two patterns stand out: many low-value homes are predicted around 1.1–1.3 rather than tracking down to their true (lower) value, and a vertical cluster of points sits at actual = 5.0, reflecting the dataset's known top-coding of MedHouseVal at 5.00001 for expensive homes — a hard ceiling the model cannot represent from MedInc and HouseAge alone. This is expected: using only two features necessarily discards location and property-size signals (latitude/longitude, room counts) that materially drive California housing prices, which caps how closely predictions can track individual actual values.

7. Bonus Experiments

7.1 Additional Hidden Layer

The original 2–5–3–1 network was compared against a deeper 2–5–3–2–1 network (one extra ReLU layer of 2 neurons), both trained with Adam at $\text{lr} = 0.001$ for 1000 epochs from seed 42:

Architecture	Hidden Layers	Test MSE
2–5–3–1 (original)	2	0.683425
2–5–3–2–1 (deeper)	3	2.761936

Table 7.1 — Effect of an additional hidden layer (Adam, $\text{lr} = 0.001$).

7.2 L2 Regularization

With the architecture and optimizer held fixed (2–5–3–1, Adam, $\text{lr} = 0.001$), a run with no weight penalty was compared against $\lambda = 0.001$:

Configuration	Final Train Loss	Test MSE
Without L2	0.666758	0.683425
With L2 ($\lambda = 0.001$)	0.666755	0.683425

Table 7.2 — Effect of L2 regularization (Adam, $\text{lr} = 0.001$).

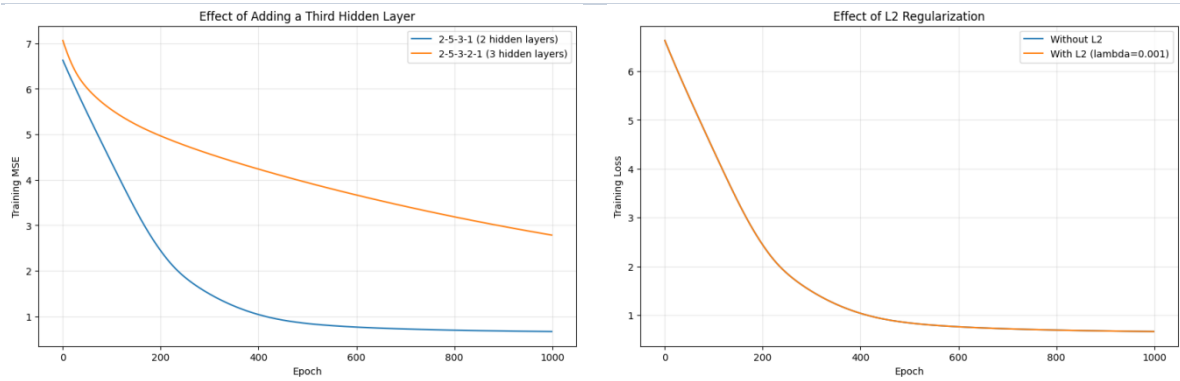


Figure 6a — Deeper network converges far slower under the same budget. Figure 6b — L2 ($\lambda = 0.001$) curve is visually indistinguishable from no L2.

Additional depth hurt performance here: the deeper network converges far more slowly under the same optimizer, learning rate, and epoch budget, and its test MSE more than quadruples ($0.683 \rightarrow 2.762$). The extra layer adds parameters and a longer gradient path without a compensating benefit for this small, two-feature problem — it would likely need more epochs or a higher learning rate to close the gap, so under this fixed budget it clearly under-performs rather than improving generalization.

L2 regularization at $\lambda = 0.001$ had a negligible effect: the training-loss curves are visually indistinguishable and the test MSE is unchanged to six decimal places. Because the inputs are already standardized and Adam keeps this small network's weights modest on its own, such a small penalty adds virtually no additional weight decay — a noticeably larger λ would be needed to see a visible regularizing effect, likely at some cost to training loss.

8. Overall Comparison & Conclusion

Configuration	Optimizer	LR	L2	Test MSE
Main grid	Momentum	0.01	0	0.619313
Main grid	Adam	0.01	0	0.625793
Main grid	Momentum	0.001	0	0.647937
Main grid	Gradient Descent	0.01	0	0.648101
Main grid	Adam	0.001	0	0.683425
Main grid	Gradient Descent	0.001	0	0.897964
Bonus — deeper (2-5-3-2-1)	Adam	0.001	0	2.761936
Bonus — L2	Adam	0.001	0.001	0.683425

Table 8.1 — All tested configurations, ranked within the main grid by test MSE.

Best-performing configuration: Momentum, lr = 0.01, architecture 2–5–3–1, no L2 — test MSE = 0.619313 (RMSE $\approx$ 0.787).

This assignment implemented a complete feed-forward regressor — forward propagation, MSE loss, manual backpropagation, and three hand-coded optimizers — entirely in NumPy, trained on two features of the California Housing dataset. Across every experiment:

- Best optimizer: **Momentum**, with Adam a close second; both substantially outperformed plain Gradient Descent at either tested learning rate.
- Best learning rate: **0.01** outperformed 0.001 for every optimizer tested, with no instability observed at the higher rate within the fixed 1000-epoch budget.
- Effect of extra depth: **negative** under this fixed budget — the 2–5–3–2–1 network converged far slower and finished with over 4 $\times$ the test MSE of the original.
- Effect of L2 regularization: **negligible** at $\lambda = 0.001$ — identical test MSE to six decimal places, since the network's weights were already small under standardized inputs and Adam.
- Overall test performance: **MSE = 0.619**, **RMSE $\approx$ 0.79**, achieved with only 2 of the dataset's 8 available features.

-
- Main limitation: MedInc and HouseAge alone omit location and size signals that drive California housing prices, and can't represent the dataset's top-coding of MedHouseVal at 5.0 — both cap achievable accuracy.